

Nachdenkzettel Beziehungen/Vererbung

1. „Class B extends X“. Jetzt fügen Sie eine neue Methode in X ein. Müssen Sie B anpassen?

Nein, wenn die Methode nicht abstrakt ist.

2. Class B extends X {

```
public void newMethodinB() { .... }
```

}

Jetzt fügen Sie eine neue public Methode in ihre abgeleitete Klasse ein. Sie möchten diese neue Methode im Code verwenden. Prüfen Sie die folgenden Codezeilen:

```
X x = new B();  
x.newMethodinB();
```

Was stellen Sie fest?

Parent class X kann nicht auf die Methoden der child class B zugreifen, da parent class X die Methoden der child class B nicht kennt

2. Class B extends X {

```
@override  
public void methodinB() { .... }
```

}

Jetzt überschreiben Sie eine Methode der Basisklasse in ihrer abgeleitete Klasse. Sie möchten diese neue Methode im Code verwenden. Prüfen Sie die folgenden Codezeilen:

```
X x = new B();  
x.methodinB();
```

Was stellen Sie fest?

Der Name der Methode bleibt unverändert, nur der Inhalt wurde abgeändert.

3. Versuchen Sie „Square“ von Rectangle abzuleiten (geben Sie an welche Methoden Sie in die Basisklasse tun und welche Sie in die abgeleitete Klasse tun)

Rectangle: getWidth(), calculateArea(), getHeight(), setWidth(), setHeight()
Square: (Override) calculateArea() setHeight(), getHeight()

4. Jetzt machen Sie das Gleiche umgekehrt: Rectangle von Square ableiten und die Methoden verteilen.

Square: setHeight(), getHeight()
Rectangle: getWidth(), setHeight(), getHeight(), setWidth()

5. Nehmen Sie an, „String“ wäre in Java nicht final. Die Klasse Filename „extends“ die Klasse String. Ist das korrekt? Wie heisst das Prinzip dahinter?

Ja, das Prinzip nennt sich Vererbung